

Modelado y Programación

Proyecto 03 : Esquema de Secreto Compartido de Shamir

Descripción del proyecto

El proyecto tiene como objetivo implementar el esquema de Secreto Compartido de Shamir para encriptar y desencriptar archivos mediante el estándar AES en modo CBC. La llave de cifrado se trata como el término independiente de un polinomio, utilizando Shamir para compartir este secreto entre un grupo de personas autorizadas. La contraseña proporcionada por el usuario se procesa a través de SHA-256 para obtener una clave de cifrado adecuada. El cifrado y descifrado de archivos se lleva a cabo utilizando AES en modo CBC a través de la interfaz criptográfica de alto nivel Fernet, asegurando así la confidencialidad de la información. La implementación se centrará en la robustez del programa, la documentación adecuada y en pruebas unitarias para cada función del programa.

Entrada del programa

El programa funciona en dos modalidades: cifrar (opción c) y descifrar (opción d).

Cifrar (opción c)

1. Nombre del archivo para guardar las evaluaciones del polinomio.
2. Número total de evaluaciones requeridas ($n > 2$).
3. Número mínimo de puntos necesarios para descifrar ($1 < tn$).
4. Nombre del archivo con el documento claro.
5. Contraseña (solicitada durante la ejecución sin hacer eco en la terminal).

Descifrar (opción d)

1. Nombre del archivo con al menos t de las n evaluaciones del polinomio.
2. Nombre del archivo cifrado.

Salida del programa

Dependiendo de cual haya sido la opción de la entrada del programa, se tendrán dos salidas distintas:

Cifrar (opción c)

1. Archivo con el documento cifrado utilizando AES en modo CBC.
2. Archivo con n parejas $(x_i, P(x_i))$ de las evaluaciones del polinomio.

Descifrar (opción d)

1. Archivo con el documento claro y con el nombre original.

Requisitos funcionales

Los requisitos funcionales son aquellos que se refieren a lo que debe hacer el programa como tal. Los que consideramos como requisitos funcionales para este proyecto son los siguientes:

- El programa debe ser capaz de generar un polinomio de Shamir con la clave de cifrado como término independiente.
- Se debe ser capaz de implementar la capacidad de evaluar el polinomio en puntos distintos para obtener las evaluaciones mínimas necesarias.
- Se debe utilizar AES en modo CBC, además, el programa debe cifrar el documento claro utilizando la clave generada a partir de la contraseña del usuario.
- El programa debe almacenar las evaluaciones del polinomio en un archivo especificado por el usuario.
- Debe ser posible reconstruir el polinomio a partir de un conjunto mínimo de evaluaciones del polinomio.
- Utilizando el polinomio reconstruido y la clave de cifrado, el programa debe descifrar el documento cifrado.
- El programa debe generar los archivos de salida según la modalidad seleccionada: documento cifrado y las evaluaciones del polinomio o el documento claro con el nombre original.
- El programa debe permitir al usuario ingresar la contraseña de manera segura, sin mostrarla en la terminal durante la ejecución del mismo.
- El programa debe operar en dos modos: cifrar (opción c) y descifrar (opción d), según la elección del usuario.
- Se deben implementar mecanismos para validar la entrada del usuario y manejar casos excepcionales, como valores inválidos o archivos no existentes.
- El programa debe estar documentado de manera clara y concisa.

Requisitos no-funcionales

Los requisitos no-funcionales son aquellos que se refieren a cómo debe comportarse el programa. Lo que esperamos lograr en el proyecto es lo siguiente:

Eficiencia: El programa debe ser constante en el manejo de imágenes, especialmente para imágenes grandes, considerando la restricción de tiempo para el procesamiento. Además de que no deberá gastar recursos extra para esa tarea.

Tolerancia a fallas: El programa debe detectar posibles fallas como la ausencia del archivo de entrada, errores de lectura de imagen, y otros problemas que puedan surgir durante la ejecución y manejarlos o erradicarlos.

Amigabilidad: En este proyecto el grupo de usuarios al que esta dirigido debe ser capaz de manejar el programa en terminal de la forma más intuitiva posible.

Escalabilidad: El programa deberá poder ser capaz de poder actualizarse de forma relativa sencilla a futuro.

Seguridad: El programa no debe realizar acciones maliciosas ni comprometer la seguridad del sistema en el que se ejecuta. Además, de ser necesario deberá poder proteger los datos privados.

Herramientas

Se decidió que se utilizará el lenguaje de programación Python, y por consecuencia su librería estándar `argparse`. Además de otras librerías que se consideraron necesarias para que el código funcione de la forma más segura y confiable posible. Las librerías que se usaron son las siguientes:

- **random**: Utilizada para la generación de coeficientes aleatorios necesarios en la construcción del polinomio de Shamir.
- **pathlib**: Empleada para el manejo eficiente de rutas y manipulación de archivos, facilitando la gestión de los archivos de entrada y salida del programa.
- **fractions (Fraction)**: Utilizada para representar y manipular fracciones, contribuyendo a la precisión en los cálculos de interpolación del polinomio.
- **cryptography.fernet (Fernet)**: Implementada para utilizar el esquema de cifrado simétrico Fernet, basado en AES en modo CBC, asegurando la seguridad y confidencialidad de la información.
- **getpass**: Empleada para solicitar y gestionar la entrada segura de contraseñas, sin mostrarlas en la terminal durante la ejecución del programa.

Para el trabajo cooperativo se utilizaron git y GitHub. Esto debido a que permiten trabajar de forma eficiente en el código, realizar un seguimiento de los cambios, gestionar problemas y realizar revisiones de código. Además, se utilizó Trello para tener un seguimiento de las tareas en su totalidad.

Pseudo-código

El pseudo-código principal con el que iniciamos y tuvimos una primera versión estable es el siguiente:

main.py

```
from polynomial import Polynomial
from getpass import getpass

shares = 10
minimum = 5

if __name__ == "__main__":
    p = Polynomial("pairs.txt")
    p.encrypt("foobar.txt", shares, minimum, getpass("Contraseña: "))
    # p.decrypt("foobar.aes")
```

password.py

```
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC
import base64

def derive_key(password: str) -> bytes:
    salt = b'\xd3\x96\xf7\xa1\x93v0\x02P4\xf6\xd6ka\x80\x84'
    kdf = PBKDF2HMAC(
        algorithm=hashes.SHA256(),
        length=32,
```

```

        salt=salt,
        iterations=480000,
    )
    return base64.urlsafe_b64encode(kdf.derive(bytes(password, 'utf-8')))
```

polynomial.py

```

from password import derive_key
from cryptography.fernet import Fernet
from fractions import Fraction
import pathlib
import random

class Polynomial:
    def __init__(self, pair_file: str) -> None:
        self.pair_file = pair_file

    def encrypt(self, clear_file: str, shares: int, minimum: int,
                password: str) -> None:
        self.write_pairs(shares, minimum, password)
        self.encrypt_file(clear_file, password)

    def decrypt(self, encrypted_file: str) -> None:
        password = self.get_password().decode('utf-8')
        self.decrypt_file(encrypted_file, password)

    def write_pairs(self, shares: int, minimum: int, password: str) -> None:
        assert shares > 2, "El número total de datos debe ser mayor a dos."
        assert shares >= minimum, "El mínimo de datos no puede ser mayor" \
            "que el número total de datos."
        coefficients: list[int] = []

    def generate_coefficients(key: bytes) -> None:
        rand = random.SystemRandom()
        coefficients.append(int.from_bytes(key))
        for i in range(minimum - 1):
            coefficients.append(rand.randint(0, 2 ** 16))

    def polynomial(x: int) -> int:
        return sum(coefficients[i] * x ** i for i in range(minimum))

    generate_coefficients(derive_key(password))

    with open(self.pair_file, 'w') as file:
        for point in range(1, shares + 1):
            file.write(f"{point} {polynomial(point)}\n")

    def get_password(self) -> str:
        with open(self.pair_file, 'r') as file:
            pairs = [(int(x), int(y))
```

```

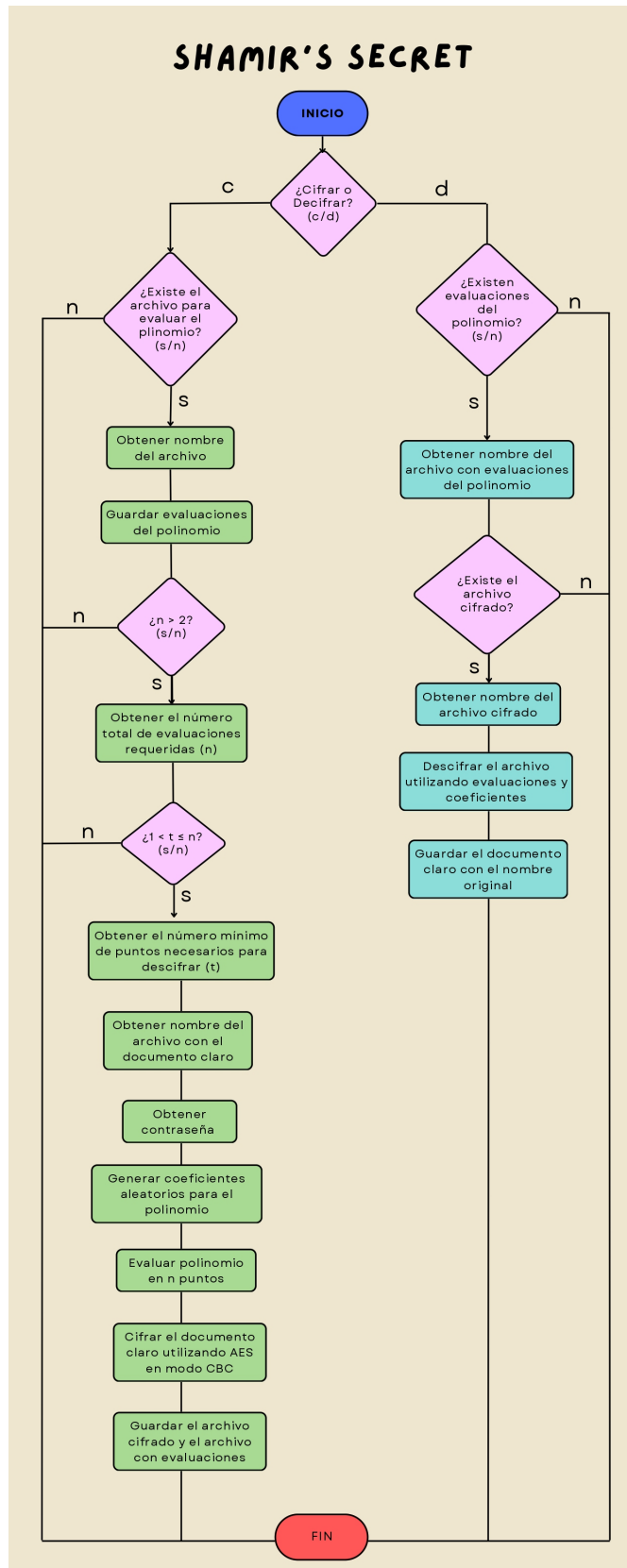
        for x, y in [line.split() for line in file]]
    acc = Fraction(0)
    for i in range(len(pairs)):
        prod = Fraction(1)
        for j in range(len(pairs)):
            if i == j:
                continue
            prod *= Fraction(pairs[i][0], pairs[i][0] - pairs[j][0])
        acc += pairs[i][1] * prod
    return int(acc).to_bytes(length=44).decode('utf-8')

def encrypt_file(self, file: str, password: str) -> None:
    clear_file = pathlib.Path(file)
    assert clear_file.exists(), "El archivo proporcionado no existe."
    fernet = Fernet(derive_key(password))
    with open(clear_file.stem + '.aes', 'wb') as encrypted_file:
        encrypted_file.write(fernet.encrypt(clear_file.read_bytes()))

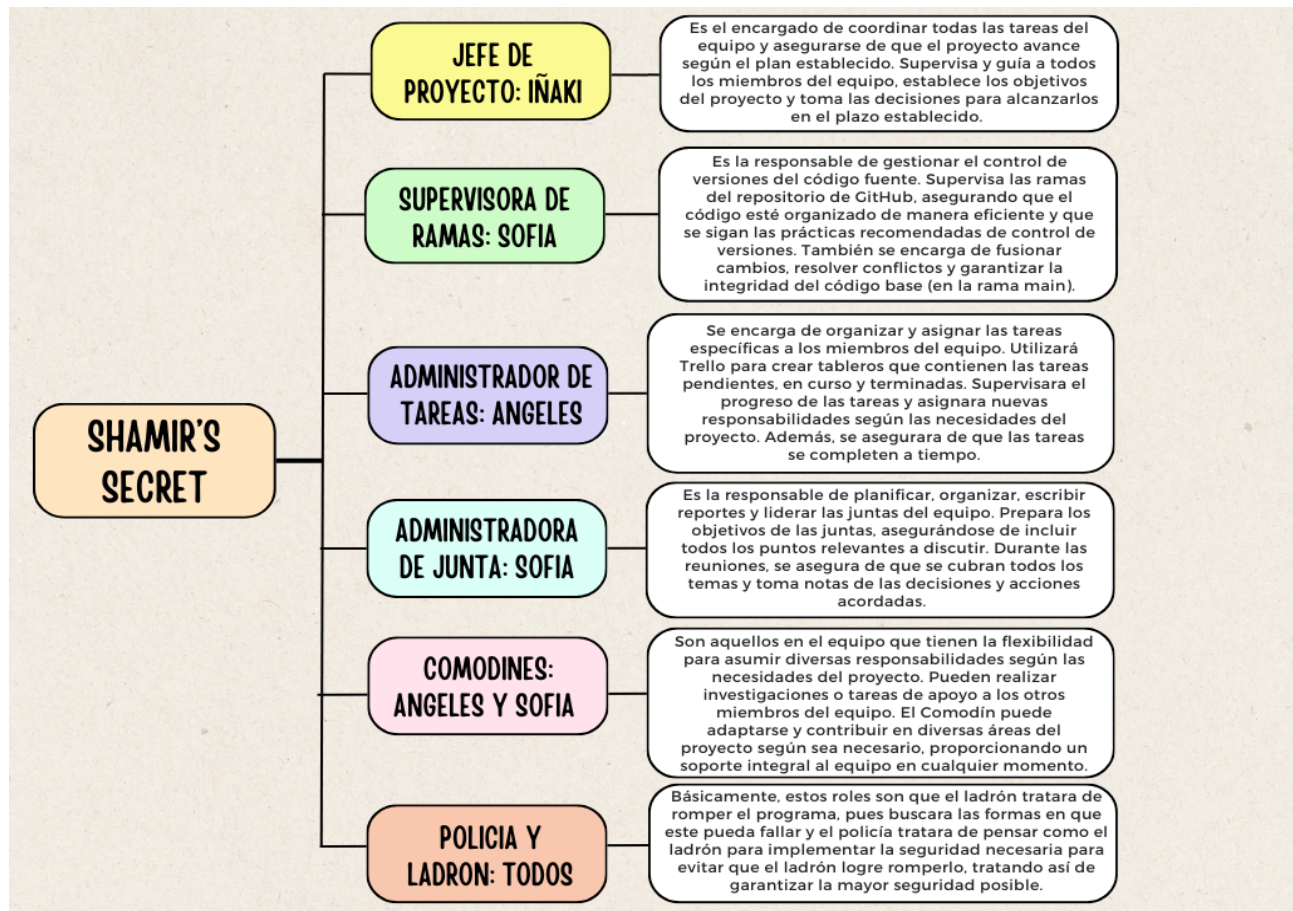
def decrypt_file(self, file: str, password: str) -> None:
    encrypted_file = pathlib.Path(file)
    assert encrypted_file.exists(), "El archivo proporcionado no existe."
    fernet = Fernet(derive_key(password))
    with open(encrypted_file.stem + '.txt', 'wb') as clear_file:
        clear_file.write(fernet.decrypt(encrypted_file.read_bytes()))

```

Diagrama de flujo



Jerarquía de roles



Resumen de juntas

Junta 1

La primer junta se realizo el miércoles, 22 de Noviembre de 5:30pm a 6:05pm. En preparación para la junta, la administradora de juntas decidió que estos serían los objetivos principales de la junta:

- Repaso, asignación de los roles ✓
- Asignación de tareas ✓
- Lluvia de ideas para el código ✓
- Revisión de pseudocódigo → Se decidió revisarlo en la siguiente junta.

A continuación se mostrara el resumen de la junta:

5:32pm - Inicia la junta 1.

5:33pm - 5:38pm

Se presenta la idea general (pseudocódigo) para poder revisarlo en equipo y se revisan los roles del equipo que se quedan como en el proyecto anterior.

5:39pm - 5:42pm

Se realiza una propuesta de la división de las tareas:

- Entrypoint.
- Clase básica de la encriptación (ya esta el esqueleto): generar pares, encriptar archivo, ... → se divide entre varios.

5:42pm - 5:46pm

Se repasaron que tareas se tienen que realizar:

- Entrada
- Encriptar archivo
- Generar y escribir pares de archivo
- Desencriptar el archivo
- Encriptar archivo
- Generar puntos del polinomios
- Tests

5:46pm - 5:59pm

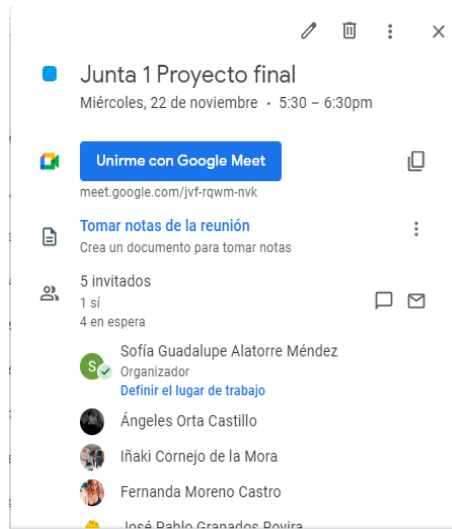
Puntos a tocar en la siguiente junta acerca del código:

- Decidir tareas (las de aparte del código).
- Asignación de que hará cada quien.

6:05pm - Se cierra la junta 1.

Anotaciones técnicas de la junta 1:

La misma se realizó de forma presencial con los tres miembros que se presentaron (Sofía, Angeles e Iñaki). Al momento no se había confirmado la participación de los otros dos miembros del equipo del proyecto anterior (posteriormente se confirmó que sólo los tres presentes serían todos los miembros del equipo). La junta se registró en el calendario de meet como si fuera virtual, pero fue presencial.



Junta 2

La segunda junta se realizó el jueves, 23 de Noviembre de 3:00pm a 3:35pm. En preparación para la junta, la administradora de juntas decidió que el objetivo principal de la junta sería checar el pseudo-código y los roles del equipo (debido a que no hubo confirmación de los otros dos miembros acerca de su situación).

A continuación se mostrará el resumen de la junta:

3:03pm - Inicia la junta 2.

3:03pm - 3:10pm

Se discute cómo se puede crear el código. Además de que se usarán objetos en el código (casi como en Java); es necesario random (se toma en cuenta que es difícil romper la aleatoriedad).

Se discute que se usará una construcción tipo iterativa, y que en secret se guardan los bits de la cadena que es la contraseña.

3:11pm - 3:17pm

Se ve que en el polinomio, se multiplicarán por cada coeficiente (número de coeficientes es el mismo de la gente), y además se discute en general cómo funciona el polinomio que está en el documento pdf dado para el proyecto.

El constructor no aceptará un número o cadena que es número, no acepta bytes. Se vio el polinomio, cómo funciona (el secreto dentro del mismo también).

3:18pm - 3:22pm

Se realiza una decisión de diseño: ¿que se cree el objeto (polinomio) y cada vez que lo evaluamos regrese los puntos o que se cree el objeto y haga todos los puntos a la vez?

Se eligió el primero por unas complicaciones de seguridad y técnicas en el segundo.

3:22pm - 3:26pm

Se siguió viendo detalles del diseño.

3:27pm - 3:34pm

Se hablo acerca de como se procedería sin los otros miembros del equipo.

3:35pm - Se cierra la junta 2.

Anotaciones técnicas de la junta 2:

La misma se realizo de forma presencial con los tres miembros que se presentaron (Sofia, Angeles e Iñaki). Al momento no se había confirmado la participación de los otros dos miembros del equipo del proyecto anterior (posteriormente se confirmo que sólo los tres presentes serían todos los miembros del equipo). La junta se no registro en el calendario de meet por un descuido de la administradora de juntas, pero si se realizo el resumen de la misma junta.

Aclaración del resto de las juntas...

Se llevaron a cabo dos juntas adicionales que no fueron registradas formalmente debido a su naturaleza de trabajo cooperativo. Estas reuniones se realizaron a través de Meet con el objetivo de colaborar en el proyecto de manera conjunta. Durante estas sesiones, se abordaron diversas tareas, incluyendo la discusión del código, la asignación de tareas, la coordinación de quién realizaría cada tarea, entre otros aspectos.

Se observó que el trabajo colaborativo durante estas reuniones era más efectivo, ya que todos los miembros del equipo estaban involucrados en varias actividades simultáneamente. A pesar de la falta de registros formales, se dedicó tiempo a trabajar en el código, la documentación, el documento PDF del proyecto, y se abordaron temas adicionales para aliviar la presión y el estrés entre los miembros del equipo.

En resumen, estas juntas adicionales fueron cruciales para avanzar en el proyecto de manera eficiente y abordar diferentes aspectos relacionados con el mismo. Aunque no se haya documentado formalmente, la colaboración activa y la distribución de tareas se llevaron a cabo durante estas reuniones de trabajo cooperativo.

Fuentes utilizadas:

- 1) Wikipedia contributors. (2023, 24 noviembre). Shamir's secret sharing. Wikipedia.
https://en.wikipedia.org/wiki/Shamir%27s_secret_sharingPreparation
- 2) Fernet (Symmetric Encryption) Cryptography 42.0.0.Dev1 documentation. (s. f.).
<https://cryptography.io/en/latest/fernet/using-passwords-with-fernet>